Chapter 3. Plan to Learn Faster

This is my friend Eric, standing in front of his backlog and task board in his team room. He’s a product owner working hard with his team to build a successful product, but right now it’s not. That doesn’t worry Eric, though. He has a strategy for making his product successful. And so far it’s working.



Eric works for a company called Liquidnet. Liquidnet is a global trading network for institutional investors. Long before Eric came to stand in front of the board in the picture, someone at his company identified a group of customers

Liquidnet could serve better, along with a few ideas of how to do that. Eric is part of a team that took those ideas and ran with them. That's what product owners do. If you thought they were always acting on their own great ideas, well, you're wrong. One of the hard parts of being a product owner is taking ownership of someone else's idea and helping to make it successful, or proving that it isn't likely to be. The best product owners, like Eric, help their entire team take ownership of the product.

Start by Discussing Your Opportunity

Eric didn't start his work by building a backlog of user stories. He started with the big idea someone had, and treated it like an opportunity for his company — because it was. He had conversations with leadership in his company to understand more. They discussed:

- *What* is the big idea?
- *Who are the customers?* Who are the companies we think would buy the product?
- *Who are the users?* Who are the types of people inside those companies we think would use the product, and what would they be using it for?
- *Why would they want it?* What problems would it solve for customers and users that they couldn't solve today? What benefit would they get from buying and using it?
- *Why are we building it?* If we build this product and it's successful, how does that help us?

Eric needed to build shared understanding with others in his organization before he could take ownership of the opportunity. He knew he was going to need to tell the story of this product many times over the coming months, so he'd better get the big stuff right now.

> Your first story discussion is for framing the opportunity.

Validate the Problem

Eric trusts his leadership's intuition, but he knows that this big idea is a hypothesis. He knows the only way to be sure the idea will succeed is when they actually see it succeed.

He first spent time talking to customers and users directly to really learn about them. Along the way he validated that there really were customers who had the problem, and they really were interested in buying a solution. Eric talked to the people who'd likely use the product. They didn't have the product today, and had only poor workarounds to address the problems the new product idea would solve.

> Validate that the problems you're solving really exist.

While Eric's been talking with customers and users, he's been building up a pool of people he thinks are good candidates to try his new software. Some companies refer to these people as *customer development partners*. Keep track of this detail, because it's going to come up later in the story.

Actually, during this stage, it wasn't just Eric. Eric was working with a small team of others who spent lots of time talking to their customers, and, in doing so, found that solving the problem wasn't so easy — and that there were other problems that needed to be solved first. The important thing for you to take away is that the more they learned, the more the original opportunity was changed — eventually, a lot. It's lucky they didn't just get to work building what they were told to. That wouldn't have served their customers or their organization.

By now Eric and his team, after talking to customers, had specific ideas for the type of solution they could build that users could use, and by doing so get the benefit their employers wanted. Now, here's where Eric and his team could have gone "all in" — where they could have bet it all. They could have built a backlog of stories that described their solution and set a team to work building it. Because they're smart people, they'd have used a story map to move

from the big idea to the specific parts to build. But, because they're *really* smart, the last thing they're going to do at this point is to build software.

Prototype to Learn

It's around here that Eric began to act as the owner for this product. He moved to envision his solution first as a bunch of simple narrative stories — *user scenarios*. Then he moved to envisioning the idea as a simple wireframe sketch. And then he created a higher-fidelity prototype. This wasn't working software. It was a simple electronic prototype, created with a simple tool like Axure, or maybe even PowerPoint.

All of these are learning steps for Eric. They help him envision his solution. Ultimately, he wants to put his solution in front of his users to see what they think. But he knows he first needs to feel confident it solves their problems before he puts it in front of them.

> Sketch and prototype so you can envision your solution.

Now, I've hidden an important detail from you. Eric was actually an interaction designer. He's the kind of designer who's used to spending time with customers and users, and used to building these simple prototypes. But, for this new product, he's also the product owner — the one ultimately responsible for the product's success. There are other product owners in Eric's company who don't have his design skills, and they very sensibly pair with designers to help with both interviewing users and envisioning solutions.

Eric did eventually bring prototypes back to users. And I wasn't there, so I don't know what really happened for Eric. But I've been in these situation lots of times, and I'm always surprised about what I learn from the people who'll really use my solution. All I can tell you is, be prepared for surprises and bad news. In fact, celebrate the bad news, because you could have received the same bad news months later, after

you’d built the software. That’s when it really sucks. Right now, it’s cheap to make changes, and you should. And Eric did.

> Prototype and test with users to learn whether your solution is valuable and usable.

After iterating his solution many times and showing it to his customers, Eric was confident he had a pretty good solution idea. Surely now he could get that backlog built, and get his team of developers to work turning that prototyped solution into real working software. But Eric’s not going to do that. Well, not exactly that. That’s a bigger bet than he’s willing to take.

Watch Out for What People Say They Want

Eric has prototyped what he believes is a viable solution. But he’s not really sure if it’s minimal — because he showed people lots of cool ideas. And if you show people all the cool ideas, of course they’ll love them. But Eric knows his job is to minimize the amount he builds and still keep people happy. How much could he take away and still have a viable solution?

Eric also knows something else that’s a bit disturbing. He knows the people who said they’d like it and use it are just guessing, too.

Think back to when you’ve bought something yourself. You may have looked at the product. You might have watched a salesperson demonstrate the cool features. You may have tried out the cool features for yourself, and you could imagine really using and loving the product. But when you bought the product and actually started using it, you found the cool features didn’t matter so much. What really mattered were features you hadn’t thought about. And, worst of all, maybe you didn’t really need the product that

much after all. OK, maybe it’s just me I’m talking about. But I’ve got lots of stuff in my garage that I wish I’d never bought.

Back to Eric. He knows his customers and users can imagine the product would be great to use, and knowing that gives him the conviction to up his bet. But the real proof is when those people actually *choose* to use it every day. That’s the real outcome he’s looking for — and the only outcome that’ll get his company the benefit it really wants. And it’s going to take more than a prototype to learn that.

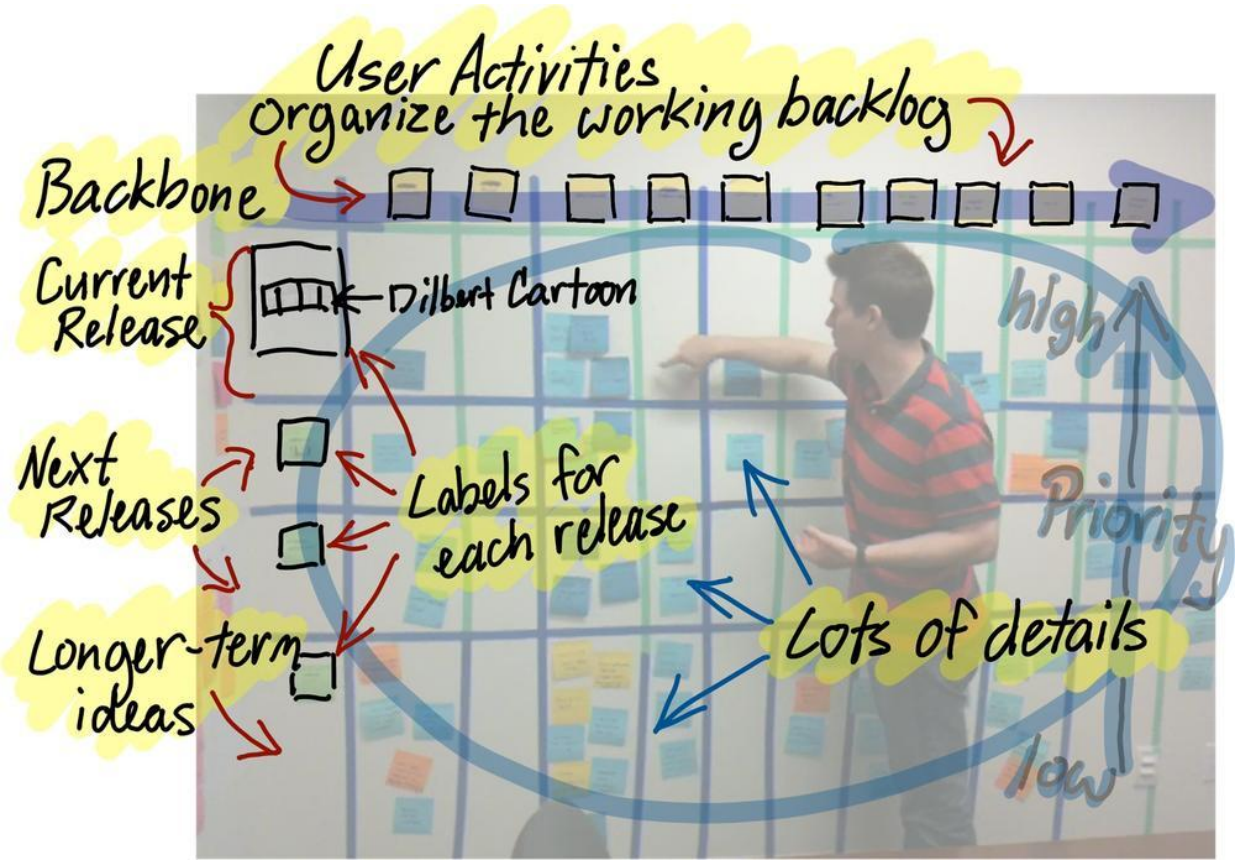
Build to Learn

Now, here’s where Eric gets to show how smart he really is.

Eric and his team actually do get to work building software. But their first goal isn’t to build a minimum viable product. Actually, it’s to build something less than minimal — just enough that potential users could do something useful with it. This is a product that wouldn’t impress too many people, and they might even hate it. It’s definitely not a product you’d want your marketing and sales people out there pitching. In fact, the only people you’d want to see this product are people who may one day use the product, and honestly care about finding a product that solves their problem.

It just so happens that Eric has a small group of people just like that. It’s the customers and users he worked with earlier when he was learning about and validating the problem. They’re his development partners. They’re the ones who gave feedback on early prototypes. And there’s a subset of them that Eric believes can best help him learn. They’re the ones he’ll put this first, less-than-minimum — and definitely not viable — product in front of. He hopes they’ll become his early adopters.

And that’s what he did.



This is Eric pointing out a slice of his current backlog. When this picture was taken, he'd already released software to his development partners. After he did, he made a point of talking to them to get their feedback. His team also built in some simple metrics so they could measure whether people were really using the software, and what they did in the software specifically.

Eric knows that people are polite. They may say they like a product, but then never use it. The "using it" is the real outcome he wants, and polite isn't helping him. Eric also knows some people are demanding. They may list all the problems the product has, or complain about bugs, but the metrics may be telling us that they use it every day anyway. And that's a good thing, in spite of all their complaining. The complaining is good, too, because it gives Eric ideas about where his next improvements should be.

Eric’s backlog is organized as a story map with the backbone in yellow stickies across the top. Those yellow stickies have short verb phrases on them that tell the big story of what his users will do in the product, but at a high level. Below it are all the details — the specific things they’ll do and need to really use the product. While the details he and his team work on change from release to release, the backbone stays pretty consistent.

The top slice, above the tapeline, is the one Eric and his team are working on right now. This release will take Eric two sprints. He’s using a Scrum development process where his sprints are two-week time-boxes. So two sprints equate to basically a month. Below that are slices running down the board. The next slice contains what they think the next release might be, and so on. To the left of each slice, just as with the Globo.com team, hangs a sticky note with the release name and a few words about what they want to learn in this release. Except for the top release, which has a *Dilbert* cartoon posted over it. It’s an inside joke in their team that I wasn’t in on.



If you look closely, the top of that current slice is sort of cleaned out. I’ve drawn in some sticky notes where they used to be. But they’re not there anymore because those

things that were on the top are the first things his team will build. As the team members worked together to plan their work, they removed those sticky notes and placed them on a task board to the right of the story-mapped backlog. That task board shows the stories they're working on now in this sprint, along with delivery task — the specific things that the developers and testers will need to do to turn the ideas in the story into working software.

One finer point of Eric's story-mapped backlog, and one that proves he's smart, is the thickness of that topmost slice. It's twice as thick as the slices below it. When Eric and his team finish a slice and deliver it to their development partners — what they call their *beta customers* — they'll move the sticky notes up from the slice below. When they do, they'll have lots more detailed discussion about this next sliced-out release. They'll play What About to find problems and fill in details. They'll talk about some of the ideas in the next release, and this discussion may result in their splitting the big idea into two or three smaller ideas. And then, they'll need the vertical height in that slice to prioritize — to make choices about what to build first.

See how smart they are?

Iterate Until Viable

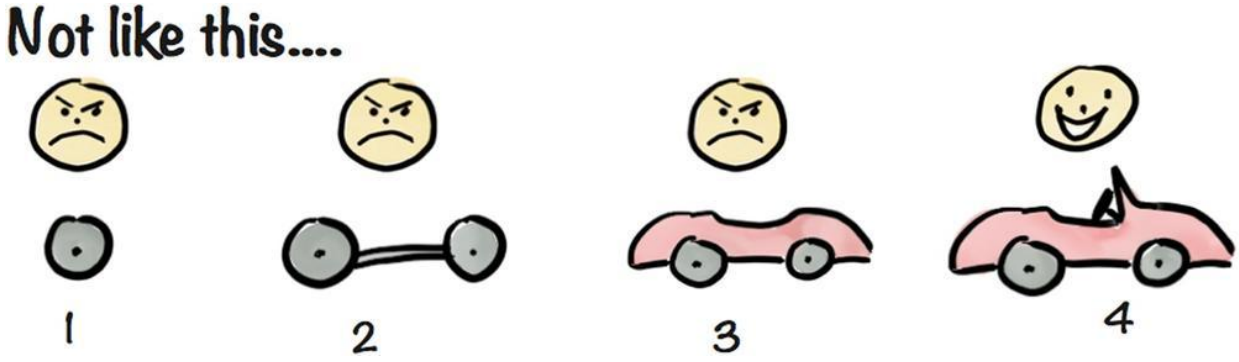
Eric may have started this whole process with an idea about what the minimum viable product might be, but he's purposely built something less than minimal to start with. He's then adding a bit more every month. He's getting feedback from his development partners — both the subjective stuff from talking to them, and the more objective stuff he gets from looking at data.

He'll keep up this strategy, slowly growing and improving the product, until his development partners actually start using the product routinely. In fact, what Eric's hoping for is that they become people who'd recommend the product to other

people — real reference customers. When they do, that's when he knows he's found minimum *and* viable. And that's when the product is safe to market and sell like crazy. If Eric and his team had tried to sell it before, they'd have ended up with lots of disappointed customers — people a lot less friendly than those with whom he built personal relationships throughout this process.

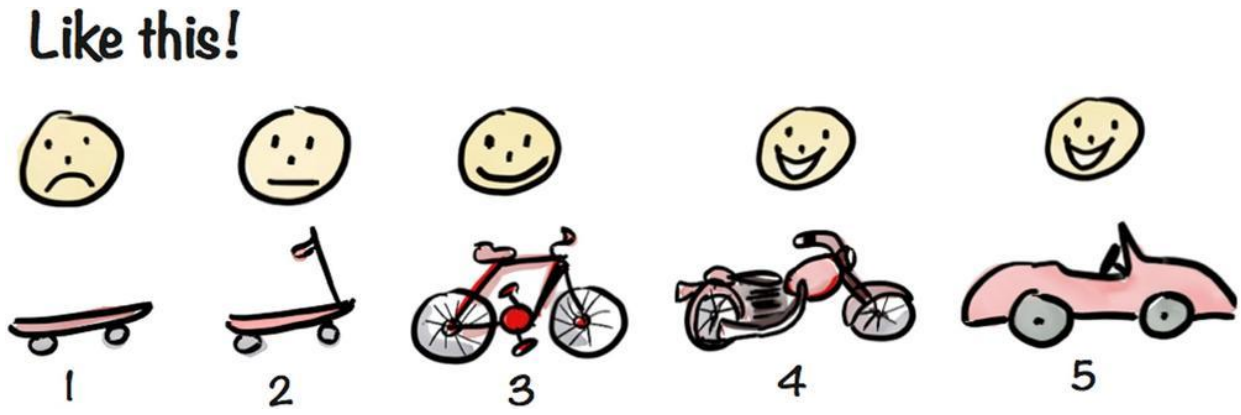
How to Do It the Wrong Way

What Eric could have done is to take his last, best prototype, break it down into all its constituent parts, and start building it part by part. Many months later, he'd have had something to release. And he'd have learned then if his big guess was right. You'll need to trust me on this, but it wouldn't have been — because it rarely is.



This is a simple visualization made by my friend Henrik Kniberg. It beautifully illustrates a broken release strategy where at every release I get something I can't use, until the last release when I get something I can.

Henrik suggests this alternative strategy:



If I plan my releases this way, in each release I deliver something people can actually use. Now, in this silly transportation example, if my goal is to travel a long distance and carry some stuff with me, and you gave me a skateboard, I might feel a bit frustrated. I'd let you know how difficult it was to travel long distances with that thing — although it was fun to goof around with it in the driveway. If your goal was to leave me delighted, you might feel bad about that. But your real goal was to learn, which you did. So that's good. You learned I wanted to travel farther, and if you picked up on it, you also learned I valued having fun.

In Henrik's progression, things start picking up at around the bicycle release because I can actually use it as adequate transportation. And, at about motorcycle level, I can *really* see this working for me — and I'm having fun too. That could be minimum and viable for me. If I really love the motorcycle thing, maybe my next best step would be a bigger, faster Harley-Davidson, and not a sports car. I'm headed for a midlife crisis right now and that Harley is sounding pretty good. But it's after I try the motorcycle, and we both learn something from that, that we can best make that decision.

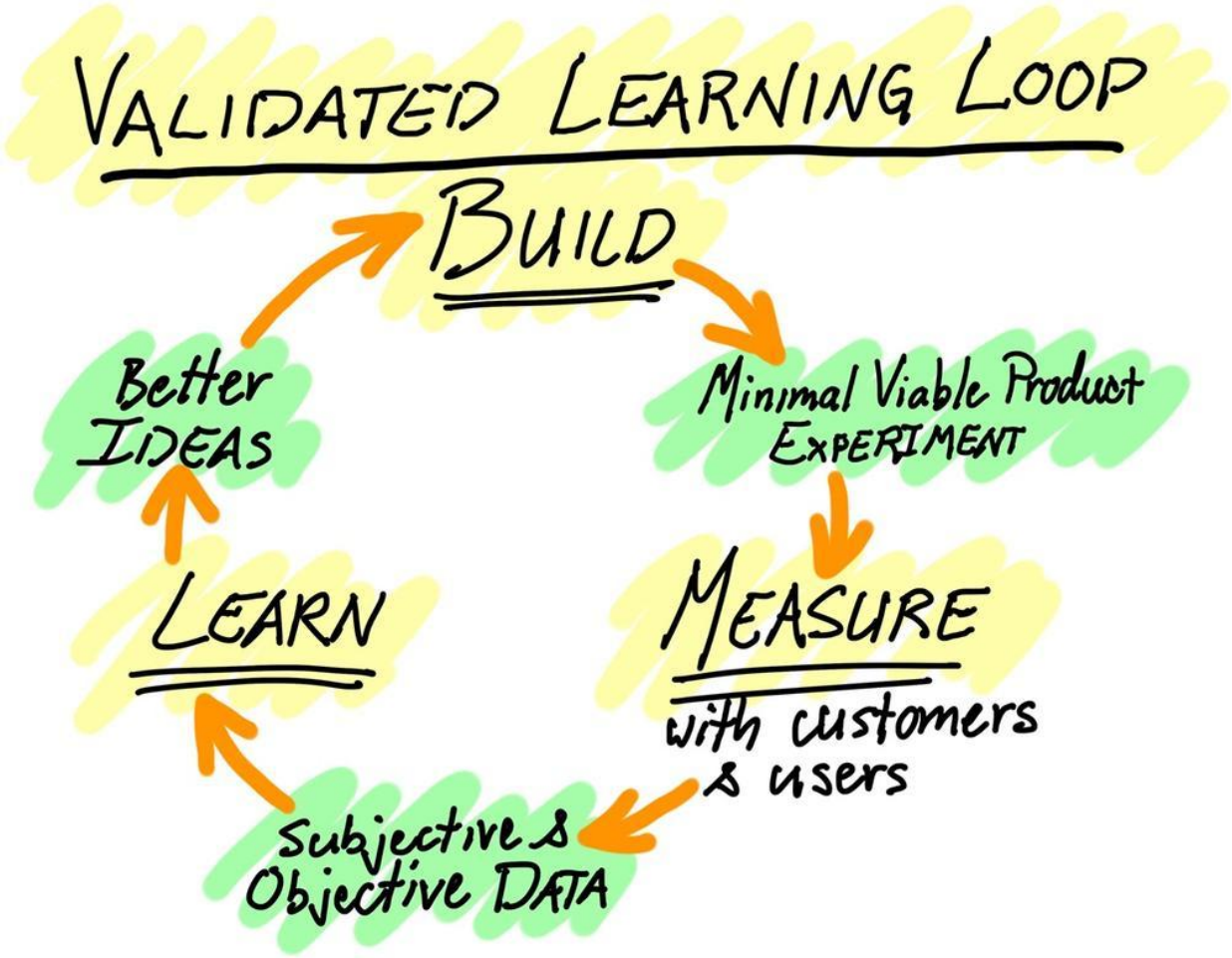
> Treat every release as an experiment and be mindful of what you want to learn.

But what about other folks who need to travel longer distance, and who have kids? For *that* target market, none of these would be good choices.

Always keep your target customers, users, and the outcomes you're hoping for in mind. It's really tough to get the same great outcome from all types of users. So focus.

Validated Learning

What my friend Eric did is apply a *validated learning strategy* — one of the important concepts in Lean Startup thinking. Eric knew that the problems he was solving, the customers and users he was solving them for, and the solutions he had in mind were all assumptions. Lots of them were pretty good assumptions. But they were assumptions just the same. Eric set out to understand the assumptions and then validate them, moving from the problems customers and users faced to the solutions he had for them. At each step he did or built something with the explicit goal of learning something.



What Eric did is the heart of the build-measure-learn loop described by Eric Ries. And by Ries's definition, each release that Eric shipped was a minimum viable product. But you can see that it wasn't viable in the eyes of his target customers and users — at least, not yet. For that reason, I like referring to Ries's MVP as a *minimum viable product experiment* — or MVPe for short. It's the smallest thing I could build to learn something. And what I learn is driving toward understanding what's really viable in the eyes of my target customers and users.

Eric used lots of tools and techniques along the way. But telling stories using words and pictures was always part of the way he worked. Using a map to organize his stories helped him keep his customers, users, and their journey in mind as he iteratively improved his product to viable.

I like using the term *product discovery* to describe what we're really doing at this stage. Our goal isn't to get something built; rather, it is to learn if we're building the right thing. It just so happens that building something to put in front of customers is one of the best ways to learn if we're building the right thing. I borrow my definition of *discovery* from Marty Cagan.[6] And my definition of *discovery* includes Lean Startup practice, Lean User Experience practice, Design Thinking practice, and loads of other ideas. And what I do during discovery continues to evolve. But the goal stays the same: to learn as fast as possible whether I'm building the right thing.

Really Minimize Your Experiments

If we recognize that our goal is to learn, then we can minimize what we build and focus on building only what we need to learn. If you're doing this well, it means that what you build early may not be production ready. In fact, if it is, you've likely done too much.

Here’s an example: when I was a product owner for a company that built software for large, chain retailers, I knew my products needed to run on a big Oracle database on the backend. But the database guys were sometimes a pain for me to work with. They wanted to scrutinize every change I made. Sometimes simple changes would take a week or more. That slowed down my team and me too much. The database guys’ concerns made sense, since all the other applications depended on that database. Breaking it was really risky for everyone. But they had a well-oiled process for evaluating and making database changes — it just took a long time.

The riskiest part for me was making sure my product was right. So we built early versions of software using simple, in-memory databases. Of course, they wouldn’t scale, and we could never release our early versions to a large general audience. But our early minimum viable product experiments (we didn’t call them that then) allowed us to test ideas with a small subset of customers and still use real data. After several iterations with customers, and after we found a solution we believed would work, we’d then make the database changes and switch our application off the in-memory database. The database guys liked us too, because they knew that when we made changes, we were confident they were the right ones.

Let’s Recap

Gary used a map to get out of the flat-backlog trap and see the big picture of his product, and then to really start focusing on who it was for and what it should be.

The teams at Globo.com used a map to coordinate a big plan across multiple teams and slice out a subset of work they believed would be a viable solution.

Eric used a map to slice out less-than-viable releases into minimum viable product experiments that allowed him to

iteratively find what would be viable.

There’s one last challenge that seems to plague software development, and that’s finishing on time. Suppose you’re confident that you have something that should be built. And suppose others are depending on it going live on a specific date. There’s a secret to finishing on time that’s been known by artists for centuries. In the next chapter, we’ll learn how to apply it to software.

[6] Marty first described what he means by *product discovery* in this 2007 essay. He later describes it in more detail in his book *Inspired: How to Create Products Customers Love* (SVPG Press).

OceanofPDF.com